diffmrc Tutorial

A hands-on walkthrough of the differentiable multireference stack

Version 0.1

This tutorial follows a user from installation to a complete state-averaged calculation with a benchmark comparison. It is paired with the *Manual*, which carries the full reference: the API, the methods and implementation chapters, the verification records, and the development architecture. The package is an early-stage research code; the tutorial states the limitations where they bite.

Contents

1 Introduction **3**
1.1 Who this tutorial is for . . . 3
1.2 What this tutorial covers . . . 3
1.3 Project status . . . 4

2 Installing and running **4**
2.1 Requirements . . . 4
2.2 Runtime settings that are not optional . . . 4
2.3 Checking the installation . . . 5

3 A first calculation **5**
3.1 The script . . . 5
3.2 What each piece does . . . 6
3.3 Reading the result . . . 6

4 State-averaged workflows **6**
4.1 A state-averaged calculation . . . 7
4.2 Reading the root table . . . 7
4.3 State weights . . . 8

5 Choosing an active space **8**
5.1 The default window, and the cases that break it . . . 8
5.2 Auditing a window in one SCF . . . 8
5.3 The corrected windows . . . 9

6 Convergence and diagnostics **9**
6.1 The criterion is the gradient, not the energy . . . 9
6.2 Multiple stationary points . . . 10
6.3 The perturbation layer is the more sensitive one . . . 10

7 A benchmark walkthrough **10**
7.1 The protocol . . . 10
7.2 The runs . . . 11
7.3 Results . . . 11
7.4 Reading the table . . . 12

8 MC-PDFT: one-step tPBE and tPBE0 **12**
8.1 A one-step calculation from the command line . . . 13
8.2 The chain, from Python . . . 13
8.3 Excited states: the on-top column . . . 13

9 Limitations and how to check them **14**
9.1 Method-family limits . . . 14
9.2 Implementation limits . . . 14
9.3 Protocol limits . . . 14

10 The RI/DF path: smaller derivative memory **15**
10.1 Build the factors . . . 15
10.2 Compare the assembled tensor with the dense one . . . 15
10.3 RI-CASCI . . . 16
10.4 Check a derivative against central differences . . . 16
10.5 The NEVPT2 comparison . . . 16
10.6 Running the examples . . . 17

11 Performance options **17**
11.1 Choosing the CI solver . . . 17
11.2 MO integral transforms . . . 18
11.3 jit with reference arrays as arguments . . . 18
11.4 Running the example . . . 19
11.5 What to record when timing . . . 19

12 CASPT2: the class table and the three contraction paths **19**
12.1 Build a reference . . . 20
12.2 The diagonal path and its class table . . . 20
12.3 How to read the class table . . . 21
12.4 The coupled full-Fock path . . . 22
12.5 The three-center path . . . 22
12.6 The N_2 case . . . 22
12.7 What to check before trusting a number . . . 23

13 Relativistic layers: scalar X2C, two-component X2C, and ECP spin–orbit **23**
13.1 Scalar DF-X2C CASSCF . . . 23
13.2 Two-component DF-X2C Hartree–Fock . . . 24
13.3 Valence-level SOMF for ECP calculations . . . 25
13.4 Where the scalar and two-component X2C energies differ . . . 25
13.5 Which layer to use . . . 26

14 RIJONX SCF: an RI-J Coulomb reference with exact exchange **26**
14.1 Closed-shell water . . . 26
14.2 Where the error comes from . . . 27
14.3 The converged gradient . . . 27
14.4 Open shell: ROHF and UHF . . . 28
14.5 ECP references . . . 28
14.6 Limits that bite in practice . . . 29

1 Introduction

`diffmrc` is an end-to-end differentiable multireference electronic structure stack. It spans configuration interaction (CASCI), complete active space self-consistent field (CASSCF), state-averaged CASSCF, and second-order N-electron valence perturbation theory (NEVPT2). Every energy, nuclear gradient, and second derivative the package produces comes out of the same code path by automatic differentiation; there is no hand-derived derivative equation anywhere in the source.

The stack is built on PySCF and pyscfad, with JAX providing the differentiable array operations. All arithmetic is double precision. The package does not replace PySCF: the Gaussian-basis integrals, the reference SCF, and the benchmark comparisons all come from PySCF and pyscfad, while the differentiable multireference chain is original to this codebase.

1.1 Who this tutorial is for

You should be comfortable with the chemistry of active spaces and with running a Python quantum-chemistry calculation from a script. You do not need prior exposure to JAX: the tutorial introduces the runtime settings that matter (`jax_enable_x64`, the CPU platform pin, and the thread budget) right where they are first needed.

1.2 What this tutorial covers

- Chapter 2: installing the package and setting up a working environment.
- Chapter 3: a complete CASSCF nuclear-gradient calculation for water, explained line by line.
- Chapter 4: state-averaged CASSCF with per-state NEVPT2, and how to read the root table it produces.
- Chapter 5: choosing an active space, with the frontier-window rules the benchmark work forced us to write down.
- Chapter 6: what the convergence diagnostics mean and how to act on them.
- Chapter 7: a full benchmark walkthrough on a real target, end to end.
- Chapter 8: the one-step MC-PDFT evaluation (tPBE and its tPBE0 hybrid), from the command line and from Python.
- Chapter 10: the resolution-of-identity path, its memory argument, and one derivative check.
- Chapter 11: the performance choices of the differentiable chain, with the measurements behind them.
- Chapter 12: the CASPT2 class table and the three contraction paths.
- Chapter 13: scalar X2C CASSCF, two-component X2C Hartree–Fock, and ECP spin–orbit.
- Chapter 14: the RIJONX SCF reference, including closed-shell, ROHF, UHF, and ECP gradients.
- Chapter 9: where the method family and the present implementation stop being reliable.

The companion *Manual* carries what a tutorial should not: the API reference, protocol tables, the design of the differentiable chain and its second derivatives, the verification records, and the development architecture.

1.3 Project status

This is a research codebase, released to accompany a methods paper and to make its results reproducible. It is not a finished library: interfaces may change without notice, test coverage is partial, and the numerical behaviour of the orbital-optimization and perturbation layers depends on convergence settings that the code reports rather than enforces. Several limitations listed in Chapter 9 are properties of the method family and will not be removed by further development of this package. Use it for research, read the convergence diagnostics it returns, and do not treat it as a black box.

2 Installing and running

2.1 Requirements

Component	Version and role
Python	≥ 3.10
numpy, scipy	required; the analysis layer needs only these
pyscfad	$\geq 0.3.3$; provides the JAX-based `gto` and `scf`
pyscf	≥ 2.5; reference comparisons and integral drivers
jax	≥ 0.4; the differentiable array layer
matplotlib	only for the figure sources

`pyscfad` pins a specific JAX version, so install the package into a dedicated environment rather than into a shared one.

```
conda create -n diffmrc python=3.11
conda activate diffmrc
pip install -e ".[core,dev]"     # kernel, tests and lint tooling
```

For work on the analysis layer alone, `pip install -e ".[dev]"` is sufficient: that layer is plain NumPy and imports nothing from the numerical kernel.

2.2 Runtime settings that are not optional

The platform and thread settings must be in place before the first JAX import; the double-precision switch immediately after it.

```
import os
os.environ['JAX_PLATFORMS'] = 'cpu'     # before importing jax
for v in ('OMP_NUM_THREADS', 'MKL_NUM_THREADS',
          'OPENBLAS_NUM_THREADS', 'NUMEXPR_NUM_THREADS',
          'VECLIB_MAXIMUM_THREADS'):
    os.environ[v] = '8'

import jax
jax.config.update('jax_enable_x64', True)
```

`jax_enable_x64` must be set: the package is validated in double precision, and single-precision runs will produce silently wrong gradients. The CPU platform pin is deliberate. The stack has been developed and verified on CPU only; on a machine with a GPU the default JAX platform would select CUDA, where this build of pyscfad raises a solver error that has no bearing on the chemistry. Pin the platform for now; GPU support is not claimed.

The thread variables keep the BLAS layer inside the thread budget the driver assumes. They are not a performance suggestion: without them a single calculation can oversubscribe a shared node.

2.3 Checking the installation

```
pytest tests/ -v        # unit and integration tests
lint-imports            # verifies the layer boundaries
```

The layer contracts are worth a word even at install time. The package is organized in six layers (kernel, analysis, policy, driver, report, command line), and the import direction between them is enforced as a test. If `lint-imports` is unavailable in your environment, `tests/test_layer_contracts.py` performs the same check. A run that passes the suite but fails the contracts is a run the developers would want to see.

Two environment notes that cost us time, recorded so that they need not cost yours:

- pyscfad's molecule object cannot be traced inside `jax.jit`; the driver layer constructs plain-array callables for anything that is jitted. If you write your own jitted wrapper around a factory from `driver.nevpt2`, use the `theta`-only factories, which are built for that context.
- The first call into a heavy node of the chain compiles an XLA program and can take minutes; subsequent calls are fast. A slow first evaluation is expected, not a hang.

3 A first calculation

This chapter computes a CASSCF nuclear gradient for water at the 6-31G level, with an active space of two electrons in two orbitals (the highest occupied and lowest unoccupied orbital of the reference). It is the smallest complete run through the kernel: a reference SCF, an orbital-rotation optimization, and one nuclear gradient from automatic differentiation.

3.1 The script

```
import os
os.environ['JAX_PLATFORMS'] = 'cpu'
for v in ('OMP_NUM_THREADS', 'MKL_NUM_THREADS',
          'OPENBLAS_NUM_THREADS', 'NUMEXPR_NUM_THREADS',
          'VECLIB_MAXIMUM_THREADS'):
    os.environ[v] = '8'

import numpy as np
import jax
jax.config.update('jax_enable_x64', True)
from jax import numpy as jnp
from pyscfad import gto as pgto, scf as pscf

from diffmrc.core.rotation import block_rotation
from diffmrc.core.orblib import orbital_pairs
from diffmrc.core.optimizer import optimize_theta
from diffmrc.core.grad import (make_energy_fn, make_grad_fn,
                               solve_theta_star, nuclear_gradient)

mol = pgto.Mole()
mol.atom = 'O 0 0 0; H 0.7557 0 0.5880; H -0.7557 0 0.5880'
mol.basis = '6-31g'
mol.build()

nmo = mol.nao_nr()
ncore, nact, nele = 4, 2, 2
pairs = orbital_pairs(ncore, nact, nmo, include_cv=True)

E_cas = make_energy_fn(ncore, nact, nele)
```

```
mf = pscf.RHF(mol)
mf.kernel()
C_rhf = jnp.asarray(np.asarray(mf.mo_coeff))

h1e = mol.intor('int1e_kin') + mol.intor('int1e_nuc')
eri = mol.intor('int2e', aosym='s1')

E_theta = lambda t: E_cas(C_rhf @ block_rotation(t, nmo, pairs), h1e, eri,
                          mol.energy_nuc())
th_star, gnorm = solve_theta_star(mol, C_rhf, E_theta, len(pairs),
                                  optimize_theta, steps=2000,
                                  g_conv=1e-12)
g = nuclear_gradient(mol, make_grad_fn(nmo, pairs, th_star, E_cas, pscf))
print(g)          # shape (natm, 3), in Eh/bohr
```

3.2 What each piece does

The reference SCF is an ordinary spin-restricted Hartree–Fock from pyscfad, whose MO coefficients are the starting point of the chain. The rotation parameter vector `t` is turned into an orbital rotation by `block_rotation`; the pair list from `orbital_pairs` includes the core–virtual rotations, which are required for a correct stationary point — omitting them leaves the optimizer stopped at a point that is not stationary. `E_cas` is the CASCI energy of the rotated orbitals.

`solve_theta_star` runs the two-stage orbital optimization and returns the parameter vector *and* the gradient norm at the point it stopped at. Do not discard `gnorm`: it is the evidence of convergence, and the package reports stationarity from this quantity, not from an optimizer status code.

`nuclear_gradient` applies automatic differentiation to the whole converged chain with respect to the nuclear coordinates, with the orbital parameters held at their converged values. The mathematics of that construction is in the Manual (Chapter on the differentiable chain); for this tutorial the point is that no derivative expression was written anywhere in the package or in this script.

3.3 Reading the result

For this geometry the gradient is small but not zero — the atoms are close to, not at, the CASSCF/6-31G equilibrium. Two checks belong to every such run:

- `gnorm`, the orbital-rotation gradient norm, should sit at or below the threshold that `solve_theta_star` was asked for. If the refinement stalls above the threshold, the returned point carries the stall in its stopping reason; treat a stalled point as a candidate, not as a result.
- The energy is invariant under rotations within a degenerate orbital pair. If a molecule has exact degeneracies, small differences between two runs with different rotation orders are a property of the chart, not a failure of the code.

The next chapter moves from the ground state to several states at once — the setting in which the state-averaged machinery and the per-state perturbation theory come in.

4 State-averaged workflows

Excited states and vertical excitation energies require the orbitals to describe several electronic states at once. The driver layer provides a state-averaged CASSCF construction whose roots are grouped into physical states, with per-state NEVPT2 corrections, all in the same differentiable chain.

4.1 A state-averaged calculation

The smallest complete example is the hydrogen molecule, whose minimal active space holds one singlet ground state, one triplet, and higher singlets.

```
import numpy as np
from jax import numpy as jnp
from pyscfad import gto, scf

from diffmrc.analysis.states import select_groups
from diffmrc.core.nevpt2.impl import cas_dets_for2
from diffmrc.core.orblib import orbital_pairs
from diffmrc.driver.nevpt2 import (make_nevpt2_sa_theta_fn,
                                   sa_nevpt2_state_table)

mol = gto.Mole()
mol.atom = 'H 0 0 0; H 0 0 0.74'
mol.basis = '6-31g'
mol.build()

nmo = mol.nao_nr()
ncore, nact, nele, nroots = 0, 2, 2, 6
pairs = orbital_pairs(ncore, nact, nmo, include_cv=True)
builder = lambda m: scf.RHF(m)
dets = cas_dets_for2(ncore, nact, nele)

comp = make_nevpt2_sa_theta_fn(mol, ncore, nact, nele, nmo, pairs,
                               builder, nroots=nroots, e2_scale=1.0)
out = comp(jnp.zeros(len(pairs)))

rows, groups = sa_nevpt2_state_table(out, dets)
for g in groups:
    print(g['multiplicity'], g['dE_casscf'], g['dE_nevpt2'])
```

The theta-only factory is the right one whenever the geometry is fixed: it builds the reference SCF and the integrals once and returns a callable of the rotation parameters alone. This is also the factory that can be wrapped in `jax.jit` for orbital optimization, because it never touches the pyscfad molecule at call time.

4.2 Reading the root table

A state-averaged eigensolver returns *roots*, not states, and the difference matters twice over.

First, spin components are degenerate. A triplet contributes three degenerate roots (the $M = +1, 0, -1$ components); a call that asks for six roots of the hydrogen molecule receives three singlets and the three components of one triplet. `sa_nevpt2_state_table` collapses energy-degenerate roots into one group per physical state, labels each group with the multiplicity read from $\langle S^2 \rangle$ of its members, and flags a group whose members disagree on $\langle S^2 \rangle$ rather than guessing.

Second, a per-component contraction can be undefined. The NEVPT2 correction of a pure $|M| = 1$ component can come out as `nan`, and the representative of the group is then chosen as the lowest component whose correction is finite. Treat a group whose entries are all `nan` as a state this protocol cannot report, not as zero.

```
triplets = select_groups(groups, multiplicity=3)   # lowest triplet
singlets = select_groups(groups, multiplicity=1, n=None)
```

4.3 State weights

By default the roots enter the state average with equal weights. When the table is to be read per state, a second pass with between-group equal weights — the components of one state share that state's weight, states share equally — stabilizes the orbitals and usually tightens the final gradient. The two-pass pattern is what the benchmark scripts run; its effect on the reported excitation energies stayed within a few tenths of an eV for the systems in this tutorial's benchmark chapter, so a single equal-weight pass is a reasonable first look.

The weights are passed as `state_weights` to either factory. The vector must have one entry per root, in root order; the helper in the benchmark chapter shows how the group table from the first pass produces it.

5 Choosing an active space

The smallest useful active space for a vertical excitation is the frontier pair: the highest occupied orbital and the lowest unoccupied one, two electrons in two orbitals (or more, when the window must widen; see below). This chapter is about the cases where that default silently selects the wrong orbitals, how to detect them, and what to do instead. The detection rules live in `diffmrc.analysis.orbitals` and cost one SCF to apply.

5.1 The default window, and the cases that break it

The default window takes the frontier pair in energy order. It is correct for the common chromophores: the carbonyl $n \to \pi^*$ pair of aldehydes and ketones, and the $\pi \to \pi^*$ pair of a simple alkene. Widening it is unnecessary there, and this tutorial's benchmark sets it as the baseline.

Two patterns break it.

A linear molecule with two pi systems. In a nitrile, a diyne, or cyanogen, the occupied pi orbitals and the virtual pi orbitals each come as degenerate perpendicular components (x and y). An energy-ordered window can pair the x component of the occupied set with the y component of the virtual set. That excitation is symmetry-forbidden, its exchange integral vanishes, and the computed excitation energy is systematically off — measured at up to 2 eV for the molecules in this tutorial's benchmark set.

An amide. In formamide the conjugated π/π^* pair sits on top of the frontier, while the lone pair that gives the characteristic $n \to \pi^*$ transition lies one orbital lower. The default window computes a $\pi \to \pi^*$ state and compares it to an $n \to \pi^*$ reference: a mismatch of more than 2 eV in the state that was supposed to be reproduced.

5.2 Auditing a window in one SCF

The audit is a per-axis decomposition of the frontier orbitals' p weight. A delocalized single-axis orbital is a pi; an atom-centred single-axis orbital is a lone pair; a window whose HOMO and LUMO sit on different axes while the occupied frontier is degenerate is a cross-component pair.

```
import numpy as np
from diffmrc.analysis.orbitals import heavy_atoms, frontier_report

# mol and mf from a plain pyscfad SCF (see the first-calculation chapter)
labels = mol.ao_labels()
C = np.asarray(mf.mo_coeff)
e = np.asarray(mf.mo_energy)
nocc = mol.nelectron // 2

rep = frontier_report(C, labels, e, nocc, heavy_atoms(labels),
                      target='npi', nelectron=mol.nelectron)
print(rep['homo']['kind'], rep['cross_component'],
```

```
    rep['suggested_window'])
```

`frontier_report` returns everything that entered the suggestion — the HOMO-1/HOMO/LUMO identities, the degeneracy map of the occupied frontier, the index of a lone pair below the HOMO when the target needs one — so that the suggestion can be checked rather than trusted. For formamide it reports that the HOMO is a delocalized pi, finds the lone pair two orbitals below, and suggests a three-orbital window spanning lone pair, pi, and pi*.

5.3 The corrected windows

For the two failure modes the recipe is the same three-orbital frontier window, three orbitals instead of two:

- **Linear two-pi molecules**: start the window one orbital lower than the default, so that both components of each degenerate pair are inside and the CI can pick the in-component combinations.
- **Amides**: start the window at the lone pair, three orbitals wide, so that the lone pair, the pi system, and pi* are all active.

One bookkeeping rule travels with the wider window: the electron count must follow the window, not be hard-coded. The scripts in this project use

$$n_{\mathrm{ele}} = N_{\mathrm{ele}} - 2\,n_{\mathrm{core}}, \tag{1}$$

which is what the three-orbital windows above need (four active electrons for the linear two-pi molecules, not two).

The effect on the numbers, from this project's benchmark records:

System	State	default window	corrected window
Cyanogen, Σ_u^- singlet	$\pi\pi^*$	+1.05	+0.62
Cyanogen, Σ_u^+ triplet	$\pi\pi^*$	+1.95	+1.20
Formamide, A'' singlet	$n\pi^*$	+2.12	+0.11

The deviations are against the reference set's best estimates (eV). The corrected formamide window reproduces the reference SC-NEVPT2 value to 0.02 eV. The residual deviations of the linear two-pi systems are a separate story — they survive every window fix and are discussed in Chapter 9.

6 Convergence and diagnostics

A multireference calculation is only as good as the stationary point it was evaluated at, and the failure modes here are quiet ones. This chapter collects what the package reports and how to read it.

6.1 The criterion is the gradient, not the energy

The orbital optimization returns both a point and the gradient norm at that point, and the norm is the verdict. The energy alone cannot serve: this project measured a run in which the energy kept descending while the parameters walked to magnitudes of 10^{13} — the model had no stationary point in the direction it was sliding along, and an energy-only stopping rule called it a success. Treat a convergence claim without a gradient norm as no claim at all.

The stopping reason distinguishes the ways a run can end:

- `CONVERGED`: the gradient reached the refinement target. This is the only reason that is a stationarity claim.
- `STALLED_GRADIENT`: the iteration stopped because a step no longer reduced the gradient. The point may still be useful, but it is a candidate, not a certificate.
- `GUARD_STOPPED`: no damped step lowered the energy. On a converged point this is the normal way to finish a Newton pass.
- `ITERATION_CAP`: the budget ran out. Re-check the gradient before using the point.

6.2 Multiple stationary points

The CASSCF energy is not convex in the orbital parameters. Different starting points can reach different stationary points whose energies differ by more than a bond energy; the water molecule at 6-31G has two minima separated by 13 kcal/mol from the same molecule and basis. Nothing in the eigensolver signals this: both points are stationary.

Practical consequences:

- Report the stationary-point spread alongside any multireference number, or run several starts and keep the lowest.
- A point with zero gradient can still be a saddle. The kernel exposes a curvature check over the Hessian (`stationary.curvature_report`); when a number matters, run it. The orbitals obtained straight from the reference SCF ($\theta = 0$) are, for the water example in this tutorial, a saddle rather than a minimum.
- Treat the spread as protocol, not error: fixed settings can still select different stationary points on different machines, and the package reports the choice it made rather than normalizing it away.

6.3 The perturbation layer is the more sensitive one

NEVPT2 depends on the orbitals more sensitively than the CASSCF energy does. Convergence gates set for the energy can leave the correction unstable: this project measured orbital-gradient stalls that moved the correction by several kcal/mol between runs whose energies agreed to the microhartree. When the target quantity is the correction,

- tighten the orbital-gradient gate rather than the energy gate, and read the gradient norm the run reports;
- confirm the correction's stability across independent runs (different starting parameters, or the two state-averaging passes of Chapter 4), instead of trusting a single converged number.

7 A benchmark walkthrough

This chapter runs the package against an external reference set and shows how to read the comparison. The reference is the QUEST database of vertical excitation energies; the protocol follows its geometry convention, and the numbers below come from this project's recorded runs.

7.1 The protocol

- **Geometries**: the reference set's own optimized geometries (bohr), never re-optimized — a geometry difference is not a method difference, and mixing the two makes the comparison unreadable.

- **Basis**: 6-31G throughout. The reference set reports its values near the complete-basis limit, so each entry carries a basis-set component that this protocol does not remove. It is visible in the deviations below and is the first thing to check when interpreting them.
- **Active space**: the frontier window of Chapter 5, widened to three orbitals for the linear two-pi and amide patterns.
- **State averaging**: equal-weight, with a second between-group pass for most systems; the three window-corrected systems below are quoted from a single equal-weight pass. The pass difference stayed within a few tenths of an eV.
- **Comparison columns**: the reference set reports both its best estimate and its own SC-NEVPT2 value. Since the package implements SC-NEVPT2, the second column isolates method-to-method differences, while the first additionally carries the reference method's own error budget.

7.2 The runs

Each target is a script under `code/`: `t315` (first set), `t317` (second set), `t321` (window-corrected re-runs), with `t316` assembling the summary table from the recorded JSON. A single run is a state-averaged CASSCF with per-state NEVPT2, followed by `sa_nevpt2_state_table` and the state selection of Chapter 4.

7.3 Results

Deviations are in eV; a positive entry means the package's excitation energy lies above the reference.

System	State	package	vs best estimate	vs SC-NEVPT2
Acetaldehyde	$^1A''$	4.58	+0.28	+0.16
	$^3A''$	4.10	+0.15	+0.07
Glyoxal	^1A_u	2.57	−0.31	−0.42
	^3A_u	2.14	−0.35	−0.45
Cyclopropenone	$^1B_1$	4.90	+0.62	+0.82
	$^3B_1$	4.48	+0.55	+0.93
Formamide	$^1A''$	5.75	+0.11	+0.02
	$^3A''$	5.44	+0.08	+0.00
Cyanogen	$^1\Sigma_u^-$	6.99	+0.62	+0.62
	$^3\Sigma_u^+$	6.11	+1.20	+1.19
Propynal	$^1A''$	6.86	+1.35	+1.28
	$^3A'$	5.36	+0.93	+0.70
Cyanoacetylene	$^1\Sigma_u^-$	7.07	+1.28	+1.24
	$^3\Sigma_u^+$	5.92	+1.47	+1.43

The mean absolute deviations, by excitation family — the split is the point of the exercise:

Family	vs best estimate	vs SC-NEVPT2
$n \to \pi^*$ (8 states)	0.31	0.36
$\pi \to \pi^*$ (6 states)	1.14	1.08

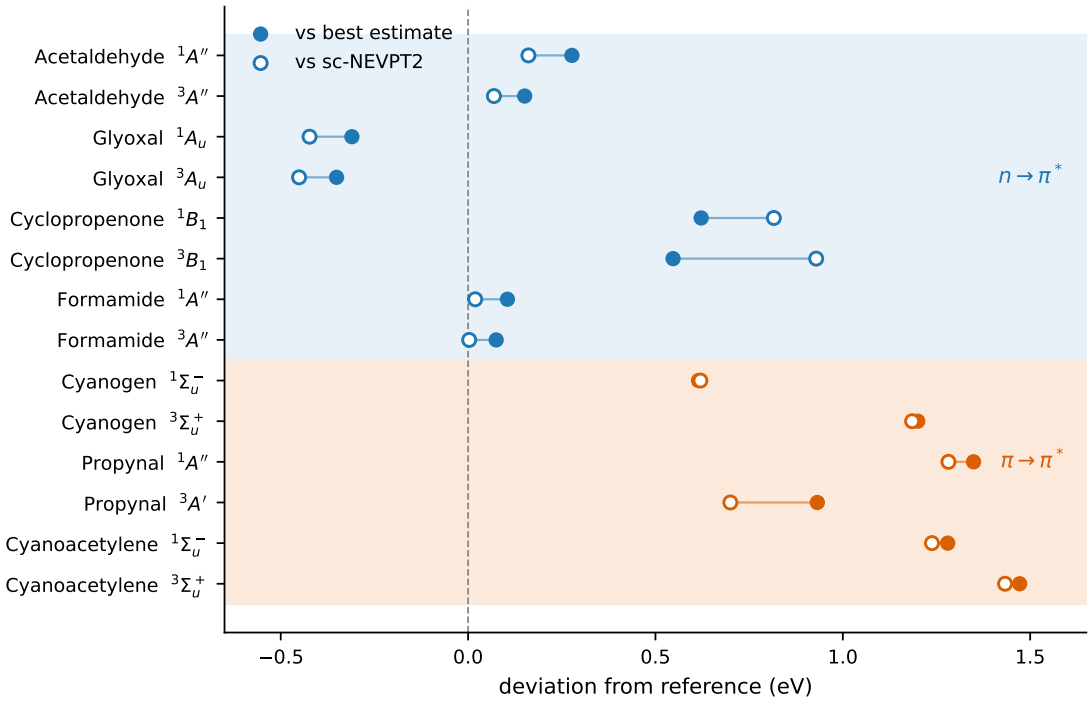


Figure 1: Deviations of the fourteen benchmark states, in eV: filled markers against the reference set's best estimate, open markers against its SC-NEVPT2 column. The bands group the two excitation families; the figure is the visual form of the two summary rows above.

7.4 Reading the table

The $n \to \pi^*$ states — the carbonyl and amide chromophores — land within about three tenths of an eV of both reference columns at this basis, which is the accuracy band the reference set itself assigns to SC-NEVPT2. The window-corrected formamide run reproduces the reference SC-NEVPT2 values to 0.02 eV.

The $\pi \to \pi^*$ states carry a uniform positive deviation of roughly 0.6 to 1.5 eV. Window corrections reduce it (cyanogen moves from +1.95 to +1.20 on its triplet) but do not remove it, and the residual has the same sign across these chemically different systems. A uniform, one-signed residual across systems is a property of the method or the protocol, not of the individual molecules; the candidate explanations — valence–Rydberg mixing at the 6-31G basis, and the multiconfigurational character of the $\pi\pi^*$ states beyond a two-orbital window — are discussed in Chapter 9. The honest summary of this benchmark is a capability map: the package's minimal-space protocol is quantitative for $n \to \pi^*$ vertical excitations and systematically high for $\pi \to \pi^*$ ones at this basis.

8 MC-PDFT: one-step tPBE and tPBE0

Multiconfiguration pair-density functional theory replaces the active-space two-electron energy of a multiconfiguration wavefunction by an on-top functional of the translated spin densities. This tutorial chapter covers the one-step protocol the driver implements: the orbitals are the CASSCF-driven ones, and the on-top energy is evaluated once at that point. This is also the protocol of the published DVS tables (Wardzala, King and Gagliardi, *J. Phys. Chem. A* 2025, `doi 10.1021/acs.jpca.4c08597`); the manual's protocol chapter carries the formulas and the conventions.

8.1 A one-step calculation from the command line

```
$ diffmrc h2.xyz --basis sto-3g --active-space 0,1 --method mcpdft
$ diffmrc h2.xyz --basis sto-3g --active-space 0,1 --method mcpdft0
```

The reported energy is the on-top total; the report carries the decomposition next to it, so the wavefunction layer stays visible:

```
on-top decomposition:
  method            mcpdft
  protocol          one-step: CASSCF-driven orbitals, one on-top
                    evaluation (tPBE)
  E_mcwfn / Eh      -0.4228339159
  E_ot    / Eh      -0.7337945639
  E_CASSCF / Eh     -1.1372838345
```

`--method mcpdft` gives the `tPBE` functional; `--method mcpdft0` gives its 0.25 hybrid, `tPBE0`, whose energy at the same orbitals is the convex combination $0.75\,E_{\mathrm{tPBE}} + 0.25\,E_{\mathrm{CASSCF}}$ (see the manual for why the hybrid collapses to this form).

The self-checks certify the wavefunction: `variational` runs on the CASSCF value, because the on-top energy is not a variational expectation value. What the checks do not certify is the on-top evaluation itself; that is carried by the anchors in the manual's verification chapter (the engine cross-check reproduces the pyscf `mcpdft` values to the last digits on H_2/STO-3G).

8.2 The chain, from Python

The two factories mirror `energy_function`: a rotation chart over the reference orbitals, the blockwise CI, then the on-top evaluation on a frozen grid.

```
import jax
jax.config.update('jax_enable_x64', True)
from jax import numpy as jnp
from pyscf import gto, scf

from diffmrc.driver.kernel import (partition_from_occupations,
                                   mcpdft_terms_function)

mol = gto.M(atom='H 0 0 0; H 0 0 0.74', basis='sto-3g')
mf = scf.RHF(mol).run(verbose=0)
space = partition_from_occupations(mf.mo_occ, [0, 1])

terms, npar = mcpdft_terms_function(mol, mf.mo_coeff, space)
E_tot, E_ot, E_mcwfn, E_cas = (float(x) for x in terms(jnp.zeros(npar)))
print(E_tot, E_ot, E_mcwfn, E_cas)          # -1.1566284798 ...
```

`terms` returns the four numbers of the report block, from one forward pass, and is differentiable in the rotation parameters exactly like `energy_function` (finite differences agree to 10^{-10} at the reference point). Passing `hyb=0.25` gives the tPBE0 combination.

8.3 Excited states: the on-top column

The vertical-excitation mode accepts the same method axis, adding a per-state one-step column to the state-averaged table:

```
$ diffmrc h2co.xyz --vee npi --nroots 2 --method mcpdft
```

The table then carries a `dE(MC-PDFT)` column in eV. One practical number to know before quoting it: the on-top column reacts to the state-averaged orbital point at first order, where

the CASSCF column reacts at second order, so it is the column that notices small orbital-point reproducibility differences between runs (the manual's limitations chapter quantifies this).

9 Limitations and how to check them

The limitations below are grouped by where they come from, because the grouping decides what to do about them: method-family limits will not be removed by further development of this package, implementation limits are measured and tracked, and protocol limits are the user's to manage. Each item comes with the check that detects it.

9.1 Method-family limits

The perturbation layer inherits the standard contracted-NEVPT2 approximations. Excitation energies are reproduced at the accuracy band the reference set assigns to SC-NEVPT2 (Chapter 7), and not better. Check: compare against a second implementation or the reference set's own NEVPT2 column before attributing a deviation to anything else.

Valence–Rydberg mixing is not resolved at small bases. $\pi \to \pi^*$ states that sit near Rydberg states — ethylene is the textbook case — shift by around an electron volt when diffuse functions enter the basis, and move again when the active window includes the diffuse pi orbitals. Check: run the target once with and once without diffuse functions; if the excitation moves by more than the claimed accuracy, the number is basis-limited, not converged.

Multiconfigurational $\pi\pi^*$ states exceed a two-orbital window. The uniform positive deviation of the $\pi \to \pi^*$ family in the benchmark chapter survives every window correction tried here. Check: widen the window by one occupied orbital and re-run; a deviation that shrinks is window-limited, one that does not is a method-family limit for this protocol.

No spin–orbit coupling and no relativistic Hamiltonian. The package works in a scalar-relativistic world with ECPs; fine structure and heavy-element spin physics are out of scope. Check: nothing to check — if the target needs SOC, this is the wrong tool.

9.2 Implementation limits

Dense diagonalization bounds the active space. The CI step builds and diagonalizes the full determinant space; spaces beyond a few thousand determinants become impractical, and iterative eigensolvers are future work.

Memory and compile time grow steeply with the parameter count. The differentiated chain broadcasts intermediates over the rotation parameters; at around five hundred parameters a single node of the Hessian path once requested a 402 GB allocation, which is why the second-order stages now run on Hessian-vector products. Expect the first evaluation of a large system to spend most of its time in XLA compilation. Check: watch the resident memory of the first optimization step, not the steady state.

CPU-only validation. The GPU path is not claimed; the platform must be pinned to CPU (Chapter 2).

9.3 Protocol limits

The convergence gate is the gradient. Review Chapter 6 before quoting any number; the stationary-point spread is part of the result.

The window rules cover the patterns met so far. The frontier audit of Chapter 5 catches the two failure modes this project hit; it is a rule set, not a guarantee, and a chemistry outside the covered patterns should be audited with the same tools rather than assumed correct.

Residual deviations carry protocol, not just method. The benchmark protocol fixes the basis at 6-31G and the geometry at the reference set's; a number quoted without those two

facts travels badly. Check: every table in this documentation states basis, geometry source, window rule, and state-weighting pass — keep that habit when exporting numbers.

10 The RI/DF path: smaller derivative memory

The nuclear-derivative path of the dense four-center integrals allocates a `nctr` × `nao`^{4} array; at `nao` = 96 that is 55 GB, and at `nao` = 175 it is 2 TB. The RI/DF layer replaces those integrals in the correlation stage by two three-center factors and a metric, and exposes a derivative path whose working set is `nao`^{2} × `naux` per atom. This chapter runs that layer on H_2O/6-31G with CAS(2,2) and ncore = 4, checks the assembly and the CI energy against the dense route, and verifies one derivative against a central difference. The manual's RI chapter carries the formulas, the full API and the error budget.

10.1 Build the factors

```
import jax
jax.config.update('jax_enable_x64', True)

from pyscf import gto, scf
from diffmrc.core import ri

mol = gto.M(atom='O 0 0 0; H 0.7557 0 0.5880; H -0.7557 0 0.5880',
            basis='6-31g', unit='Angstrom', verbose=0)
mf = scf.RHF(mol).run(verbose=0)
auxmol, label = ri.make_auxmol(mol)
factors = ri.ri_factors(mol, auxmol)

print(label, factors.nao, factors.naux, factors.cond)
# def2-universal-jkfit 13 113 2.47e+07
```

`make_auxmol` tries the fitting bases in `ri.AUX_BASIS_PRIORITY` and reports which one it used; the returned label is what should be quoted with any RI number. On this system the metric condition number is 2.5×10^7 and no direction is dropped at the default `rcond` = 10^{-12}.

10.2 Compare the assembled tensor with the dense one

```
import numpy as np

eri_ri = ri.ri_eri_ao(mol, auxmol, factors=factors)
eri_dense = mol.intor('int2e', aosym='s1')

bf = factors.b.reshape(-1, factors.naux)
ref = bf @ np.linalg.solve(factors.s, bf.T)
print('identity', np.abs(eri_ri - ref.reshape(eri_ri.shape)).max())
print('rms     ', np.sqrt(np.mean((eri_ri - eri_dense) ** 2)))
print('max     ', np.abs(eri_ri - eri_dense).max())
```

The identity check compares the two-step GEMM assembly with an explicit metric solve; it is the implementation error and should print at the 10^{-12} level. The direct comparison with the dense tensor is the RI approximation itself and is four to eight orders of magnitude larger. On this run:

```
identity 1.068e-12
rms      8.317e-06
max      7.967e-04
```

10.3 RI-CASCI

The MO path transforms the three-center tensor once and forms the four-center MO integrals by one GEMM over the auxiliary index:

```
from diffmrc.core.casci import cs_casci

c = mf.mo_coeff
h1_mo, eri_mo_dense = mo_integrals(c, mf.get_hcore(), eri_dense)
b_mo, eri_mo_ri = ri.ri_eri_mo(c, factors.b, factors.jhalf)

kw = dict(ncore=4, nact=2, nele_act=2, return_ci=True)
e_dense, ci_dense = cs_casci(h1_mo[:6, :6], eri_mo_dense[:6, :6, :6, :6], **kw)
e_ri, ci_ri = cs_casci(h1_mo[:6, :6], eri_mo_ri[:6, :6, :6, :6], **kw)
print(float(e_ri) - float(e_dense))
# +1.054e-05
```

`mo_integrals` is the four-step two-operand transform defined in the example script. The RI shift of the CASCI energy is 1.1×10^{-5} Eh (0.007 kcal/mol), and the CI vectors overlap to ten digits. At this size the shift is dominated by the fitting-basis error, not by the contraction.

10.4 Check a derivative against central differences

```
def displaced(mol, atom, axis, step):
    coords = mol.atom_coords(unit='Bohr').copy()
    coords[atom, axis] += step
    symbols = [mol.atom_pure_symbol(i) for i in range(mol.natm)]
    return gto.M(atom=[(s, c) for s, c in zip(symbols, coords)],
                 basis=mol.basis, unit='Bohr', verbose=0)

step = 1e-4
for atom in range(mol.natm):
    d3 = ri.ri_three_center_deriv(mol, auxmol, atom)
    for axis in range(3):
        mp = displaced(mol, atom, axis, step)
        mm = displaced(mol, atom, axis, -step)
        ap, _ = ri.make_auxmol(mp)
        am, _ = ri.make_auxmol(mm)
        fd = (ri.ri_three_center(mp, ap)
              - ri.ri_three_center(mm, am)) / (2 * step)
        print(atom, axis,
              np.abs(d3[axis] - fd).max() / np.abs(fd).max())
```

The auxiliary molecule is rebuilt at each displaced geometry so that the fitting basis follows the nuclei. The worst relative deviation over all atoms and axes is

```
worst relative deviation   3.641e-09  (atom 0, axis 0)
```

The same comparison for the assembled ERI, `ri_eri_deriv` against a central difference of `ri_eri_ao`, gives 3.7×10^{-8} at the same step. Part of that residual is the finite-difference truncation amplified by the metric inverse square root; the manual's error budget separates the two contributions.

10.5 The NEVPT2 comparison

`examples/ri_nevpt2.py` feeds the RI MO integrals to `core.nevpt2.impl.nevpt2_e2_full2` and compares the correction with the dense route at the same orbitals. On H_2O/6-31G, CAS(2,2), ncore = 4:

```
e2 dense              -0.1276536999 Eh
e2 RI, same CI        -0.1276366317 Eh
e2 RI, end to end     -0.1276366150 Eh
RI - dense, same CI   +1.707e-05 Ha
RI - dense, end to end +1.708e-05 Ha
```

The three numbers separate the integral approximation (same CI) from the small CI shift caused by it (end to end). The absolute correction is that of a fixed-orbital calculation, not a production NEVPT2 value; what the example validates is the RI-minus-dense difference.

10.6 Running the examples

From the repository root, with the package importable:

```
$ python examples/ri_df.py
$ python examples/ri_nevpt2.py
# if the package is not installed:
$ PYTHONPATH=src python examples/ri_df.py
```

Both scripts set `JAX_PLATFORMS=cpu` if it is unset and require only `pyscf`, `jax`, `numpy` and `diffmrc`. On the reference machine, `ri_df.py` takes 6.4 s of script time (8.0 s wall, 0.9 GB peak RSS) and `ri_nevpt2.py` takes 36 s of script time (40 s wall, 1.6 GB peak RSS); the NEVPT2 part is dominated by the first JAX compilation.

11 Performance options

Four settings decide most of the cost of a run, and only one of them can change a number the user reads. This chapter shows how to set each one and what to expect. The measured anchors behind the recommendations are in the Manual (Chapter on performance and numerical hygiene); the example script reproduces the routing and the two implementation comparisons on a small system.

11.1 Choosing the CI solver

`canonicalize_jax` and `canonicalize_sa_jax` take a `ci_solver` argument with three values:

- `'auto'` (default): the solver is chosen from the determinant count $N_{\mathrm{det}} = \binom{2n_{\mathrm{act}}}{n_{\mathrm{ele,act}}}$. At or above 2000 determinants the blocked solver is used, below it the plain solver.
- `'plain'`: the original inverse-iteration solver. It is faster in eager execution on small spaces and returns a CI vector with a residual of order 10^{-8}.
- `'blocked'`: the S_z-blocked solver. It returns the exact eigenvector and works one block at a time, which is what makes large determinant spaces fit in memory.

```
from diffmrc.core.nevpt2.impl import canonicalize_jax, resolve_ci_solver

# What would 'auto' select for this active space?
solver = resolve_ci_solver('auto', nact, nele_act)
print(solver)          # 'plain' below 2000 determinants, else 'blocked'

C_can, ci_can, eps = canonicalize_jax(
    C, h1e, eri, ci, ncore=ncore, nact=nact, nele_act=nele_act,
    ci_solver='auto')
```

Expected behaviour on the two measured anchors:

Active space	N_{det}	`'auto'` selects	Why
CAS(6,6)	924	`'plain'`	blocked is 1.8× slower eager with no memory saving
CAS(8,8)	12870	`'blocked'`	peak memory 37.83 → 6.78 GB, and the plain jit graph does not compile

The plain solver is not a smaller version of the blocked one: the orbitals and orbital energies are bit-identical between the two, and the CI vector differs only by a global phase and by the plain solver's inverse-iteration residual. A user who needs the exact pre-change CI vector can pass `'plain'`; a user who forces `'blocked'` on a small space pays a real cost, measured at 1.8 times on 924 determinants and 36 percent on the whole NEVPT2 theta chain at CAS(2,2).

The flag affects only the internal re-solves of the canonicalization routines. The main CI solve of the production chain stays on the plain solver, because substituting the blocked eigenvector there would move the NEVPT2 correlation energy beyond the 10^{-12} acceptance band. In the state-averaged routine with more than one root the flag has no effect, because no multi-root blocked solver exists.

11.2 MO integral transforms

Every MO integral transform in the differentiable chain goes through `core.nevpt2.impl.stepwise_mo_ints`. Use it in place of the one-shot four-operand contraction:

```
from diffmrc.core.nevpt2.impl import stepwise_mo_ints

# Use this:
h1e_mo, eri_mo = stepwise_mo_ints(C, h1e, eri)

# Not this:
# h1e_mo = jnp.einsum('ai,ab,bj->ij', C, h1e, C)
# eri_mo = jnp.einsum('pi,qj,rk,sl,pqrs->ijkl', C, C, C, C, eri)
```

The two forms are algebraically the same. On the release tree the stepwise form reproduces the H2O/6-31G CAS(2,2) energy and theta gradient bit for bit, and the largest difference over the checked systems is 5.7×10^{-14} Ha on the linear CO case. On a 96-orbital physical integral array it is also eleven to thirteen times faster (5.1–5.8 s down to 0.42–0.47 s). On a small array both forms finish in microseconds, so the gain is a production-size effect rather than a small-system one.

The reason to keep the four steps is not only speed. The one-shot form was measured at an absolute accuracy of about 10^{-6} on the large array, and a linear molecule's degenerate Fock pairs have gaps near 10^{-15}; the perturbation rotates the eigenvector basis inside the degenerate subspace, which changed the theta gradient by eight percent and made the Hessian-vector product NaN.

11.3 jit with reference arrays as arguments

The theta-only factories accept the reference arrays either as closure constants or as runtime arguments. Closure constants are compiled into the traced graph, so a jitted wrapper must recompile for every geometry. Passing the arrays as arguments keeps one executable for all geometries whose arrays share a shape and dtype.

The callable returned by `make_nevpt2_theta_fn` exposes the array-argument function as `component_args` and the tuple to pass as `arrays`:

```
import jax
```

```
E_theta = make_nevpt2_theta_fn(mol, ncore, nact, nele, nmo, pairs,
                               scf_builder)
E_jit = jax.jit(E_theta.component_args)     # one compilation
E_ref = E_jit(theta, *E_theta.arrays)       # works for every geometry

# The orbital-point search accepts the same pair:
converge_orbital_point(E_theta.component_args, len(pairs),
                       arrays=E_theta.arrays)
```

On three geometries (the reference scaled by -1, 0, and $+1$ percent) the closure form made three compilations and took 8.60 s for the three calls; the array form made one compilation and took 2.67 s. At f-block window sizes a single compile is of order fifteen minutes, which is where the difference matters.

The joint geometry-dependent entry `make_nevpt2_joint_fn` cannot be traced under `jax.jit` in this pyscfad build because the SCF inside the call raises a tracer-conversion error. The array form is therefore for the fixed-geometry theta-only entry; the NEVPT2 correlation term is inside that jitted function and agrees with its eager evaluation bit for bit.

11.4 Running the example

The script `examples/performance_options.py` in the repository demonstrates all four points on a small system. From the repository root:

```
PYTHONPATH=src python examples/performance_options.py
```

It prints the routing decision for 924 and 12870 determinants, the maximum difference between the stepwise and one-shot transforms on a random 10^4 array, the H2O/6-31G CAS(2,2) comparison of the three solver settings, and the compile counts of the two jit forms. The recorded run ends with:

```
CAS(6,6)  ndet=   924  auto -> plain
CAS(8,8)  ndet= 12870  auto -> blocked
max |d h1|  = 0.000e+00
max |d eri| = 1.332e-15
auto     vs plain: max|dC| = 0.0e+00  max|deps| = 0.0e+00  |dE_CAS| = 0.0e+00  max|dci|
     = 0.0e+00
blocked  vs plain: max|dC| = 0.0e+00  max|deps| = 0.0e+00  |dE_CAS| = 0.0e+00  max|dci|
     = 4.7e-10
array-argument form: jit objects = 1, compilations = 1
closure form:        jit objects = 3, compilations = 3
```

The H2O energy is -61.314092285778507 Eh in all three solver settings; the blocked CI vector differs from the plain one by 4.7×10^{-10} after phase alignment, which is the plain solver's inverse-iteration residual and not a solver error.

11.5 What to record when timing

Record the compilation count and the peak resident memory together with any wall clock. The first two are load-independent and decide whether a run is feasible; wall clock on a shared node depends on the load average and is only a relative quantity. The example script prints the compile counts from the jit objects' cache size, and the Manual chapter records the peak memory of each route at the two determinant anchors.

12 CASPT2: the class table and the three contraction paths

The perturbation layer reports its energy class by class: `V0_ijab`, `Vm1_iab`, `Vm2_ab`, `V1_ija`, `V2_ij`, `V0_ia`, `Vm1_a`, and `V1_i`. Reading that table is the difference between knowing that a

number is close to a reference and knowing which part of the wavefunction produced it. This chapter builds a small reference, runs the three contraction routes, and shows how to compare them.

12.1 Build a reference

The reference chain is RHF, CASSCF, state-specific CASCI, then canonicalization. The canonicalization is not cosmetic: the CASPT2 denominators are defined in the semicanonical basis, and the active-natural-orbital rotation is part of the definition.

```
import numpy as np
import jax
jax.config.update('jax_enable_x64', True)
import jax.numpy as jnp

from pyscf import gto, scf, mcscf
from diffmrc.core.casci import cs_casci
from diffmrc.core.nevpt2.impl import canonicalize_jax, cas_dets_for2

ncore, nact, nele = 4, 2, 2
nocc = ncore + nact

mol = gto.M(atom='O 0 0 0; H 0.7557 0 0.5880; H -0.7557 0 0.5880',
            basis='6-31g', unit='Angstrom', verbose=0)
mf = scf.RHF(mol).run(verbose=0)
mc = mcscf.CASSCF(mf, nact, nele)
mc.kernel()

C = jnp.asarray(mc.mo_coeff)
h1e = jnp.asarray(mol.intor('int1e_kin') + mol.intor('int1e_nuc'))
eri = jnp.asarray(mol.intor('int2e', aosym='s1'))

def mo_ints(C, h1e, eri):
    h1 = jnp.einsum('ai,ab,bj->ij', C, h1e, C)
    z = jnp.einsum('pi,pqrs->iqrs', C, eri)
    z = jnp.einsum('qj,iqrs->ijrs', C, z)
    z = jnp.einsum('rk,ijrs->ijks', C, z)
    return h1, jnp.einsum('sl,ijks->ijkl', C, z)

h1_mo, eri_mo = mo_ints(C, h1e, eri)
_, ci = cs_casci(h1_mo[:nocc, :nocc],
                 eri_mo[:nocc, :nocc, :nocc, :nocc],
                 ncore=ncore, nact=nact, nele_act=nele,
                 nroots=1, return_ci=True)
C2, ci2, eps = canonicalize_jax(C, h1e, eri, ci, ncore=ncore,
                                nact=nact, nele_act=nele)
h1_mo2, eri_mo2 = mo_ints(C2, h1e, eri)
dets = cas_dets_for2(ncore, nact, nele)

print('CASSCF', mc.e_tot, 'CI norm', float(jnp.linalg.norm(ci2)))
print('eps', np.asarray(eps))
```

`dets` is a list of determinant bitmaps with the core spin orbitals first. `eps` is the semicanonical Fock diagonal; it must not be replaced by the RHF orbital energies.

12.2 The diagonal path and its class table

```
from diffmrc.core.caspt2 import caspt2_contracted_e2
```

```
e2, breakdown = caspt2_contracted_e2(
    np.asarray(h1_mo2), np.asarray(eri_mo2), np.asarray(ci2), dets,
    ncore=ncore, nact=nact, nele=nele, eps=np.asarray(eps))

for name, value in breakdown.items():
    print(f'{name:10s} {value: .10f}')
print('total', e2)
```

For H_2O/6-31G CAS(2,2) the two three-active-index classes `Vm1_a` and `V1_i` are below 10^{-9} Eh: they vanish on a two-orbital active space. The five double and single classes and the `V0_ia` block carry the correction. On CO/6-31G CAS(2,2) the same structure holds with different weights.

12.3 How to read the class table

For a class c with energy $e_{2,c}$ and total e_2, the two useful scalars are the signed share and the absolute share,

$$s_c = \frac{e_{2,c}}{e_2}, \qquad a_c = \frac{|e_{2,c}|}{\sum_{c'} |e_{2,c'}|}. \tag{2}$$

The signed share says whether the class lowers or raises the energy relative to the total; the absolute share says which class dominates the correlation. A large signed share with a small absolute share is possible when the total is small. A cancellation index

$$C = \frac{\sum_c |e_{2,c}|}{|e_2|} \tag{3}$$

near one means the classes add coherently; $C \gg 1$ means the total is a cancellation between classes and the total alone is not a safe diagnostic.

The point is not academic. On N_2/6-31G CAS(6,6) the default-path total is 0.989 of the reference, but the class residuals are `V1_ija` +55%, `V2_ij` −76%, `V0_ia` +10%, and `Vm2_ab` +12%. The total is close for the wrong reason; the class table is what identifies the cancellation. On the CAS(2,2) anchors the class ratios are all within a few percent of one, so the total is a meaningful summary there.

The denominator sign matters too. A class with $A - E_0 S > 0$ lowers the energy; a class with $A - E_0 S < 0$ would raise it. The code skips a denominator with $|A - E_0 S| < 10^{-10}$; a skipped class is reported as zero in the breakdown, not as a missing key. When a class is unexpectedly zero, print the denominator and the overlap eigenvalue before attributing it to symmetry.

The default `class_diagonal='function'` evaluates each contracted function with its own denominator. When the class overlap has degenerate eigenvalues the diagonal reading depends on the basis chosen inside the degenerate block. The deterministic gauge makes the number reproducible, but the class energy is invariant only under `class_diagonal='block'`:

```
e2_block, bd_block = caspt2_contracted_e2(
    np.asarray(h1_mo2), np.asarray(eri_mo2), np.asarray(ci2), dets,
    ncore=ncore, nact=nact, nele=nele, eps=np.asarray(eps),
    class_diagonal='block', v0_ia='class')

for name in breakdown:
    print(f'{name:10s} default {breakdown[name]: .8f}'
          f'   block {bd_block[name]: .8f}')
```

On the CAS(2,2) anchors the block and per-function readings lie close together. The documented machine-precision agreement is between the per-function reading and the orthonormalized diagonal (`class_diagonal='ortho'`); on larger active spaces the readings separate, and the N_2 case in the manual chapter is the one that separates them.

12.4 The coupled full-Fock path

```
from diffmrc.core.caspt2 import caspt2_contracted_e2_coupled

e2_cpl, bd_cpl, system = caspt2_contracted_e2_coupled(
    np.asarray(h1_mo2), np.asarray(eri_mo2), np.asarray(ci2), dets,
    ncore=ncore, nact=nact, nele=nele, return_system=True)

print('coupled', e2_cpl)
print('labels ', system['labels'])
print('E0     ', system['E0'])
print('x norm ', float(np.linalg.norm(system['x'])))
print('min eig(S)', float(np.min(np.linalg.eigvalsh(system['S']))))
```

The coupled path keeps the inter-class coupling of the state-specific generalized Fock operator. It is a NumPy value layer; it does not replace the differentiable diagonal path. The `system` dictionary is the object to inspect when a class value is questioned: the overlap spectrum, the reduced solution `x`, and the class labels are all there.

12.5 The three-center path

The three-center contraction consumes the metric-folded factor `u_mo` instead of the four-center tensor:

```
from diffmrc.core import ri
from diffmrc.core.caspt2_three_center import (
    caspt2_contracted_e2_three_center)

auxmol, label = ri.make_auxmol(mol)
factors = ri.ri_factors(mol, auxmol)
u_mo, _ = ri.ri_eri_mo(np.asarray(C2), factors.b, factors.jhalf,
                       full=False)

e2_3c, bd_3c = caspt2_contracted_e2_three_center(
    np.asarray(h1_mo2), u_mo, np.asarray(ci2), dets,
    ncore=ncore, nact=nact, nele=nele, eps=np.asarray(eps))

print(label)
print('three-center', e2_3c)
print('max class difference',
      max(abs(bd_3c[k] - breakdown[k]) for k in breakdown))
```

The two paths use the same class definitions and the same relative weight between the two orderings, so the class difference is at the roundoff level. The three-center path is not a different approximation: it computes the same CASPT2-D energy with the RI factorization of the integrals. The RI error enters through `u_mo`, as it does in the RI-CASCI chapter.

12.6 The N_2 case

The system that exercises the three-active-index classes is N_2/6-31G CAS(6,6). It needs a converged CASSCF with the NR orbital step and a tight gradient tolerance; the reference values used by the validation are RHF -108.867763375101, CASSCF -109.01554685351947, CASPT2-D -0.071520733, and full-FIC -0.07301141705494 Eh. The driver in `scripts/caspt2_n2_validation.py` splits the reference build and the two expensive contractions into separate stages:

```
python scripts/caspt2_n2_validation.py --stage ref --outdir out
python scripts/caspt2_n2_validation.py --stage default --outdir out
```

```
python scripts/caspt2_n2_validation.py --stage coupled --outdir out
```

The default-path ratio to the frozen ORCA CASPT2-D reference is 0.989; against the release reference at one BLAS thread it is 0.983. The two three-active-index classes contribute −0.003648 and −0.001625 Eh. The coupled path reaches 1.0232. The residual is a class-space difference in `V1_ija`, documented in the manual chapter, not a convergence artifact.

12.7 What to check before trusting a number

- The CI vector is state-specific and matches the determinant list.
- `eps` comes from `canonicalize_jax`, not from the SCF.
- The class table is read, not just the total. A total that looks close to a reference can be a cancellation between classes with opposite errors.
- When a class value enters a decision, use `class_diagonal='block'` and `v0_ia='class'`; the default diagonal is gauge dependent on degenerate class overlaps.
- The three-center path needs the metric-folded factor; the raw `ri_three_center` tensor is not the input.

13 Relativistic layers: scalar X2C, two-component X2C, and ECP spin–orbit

This chapter runs the three relativistic entry points that do not belong to the differentiable multireference chain: the scalar (spin-free) density-fitted X2C CASSCF, the two-component density-fitted X2C Hartree–Fock, and the valence-level spin–orbit mean field for effective core potentials. All three are NumPy or pyscf layers; none runs on pyscfad and none contributes to the JAX derivative chain.

13.1 Scalar DF-X2C CASSCF

The scalar layer transforms the one-electron Hamiltonian with the spin-free Foldy–Wouthuysen operator and keeps the non-relativistic Coulomb interaction, density fitted. The wavefunction is a real spin-adapted determinant expansion, so the CASSCF machinery is the same as in the non-relativistic chapters.

```
import numpy as np
from pyscf import gto
from diffmrc.core import x2c_casscf_df as xc

mol = gto.M(atom='O 0 0 0; H 0 -0.757 0.587; H 0 0.757 0.587',
            basis='6-31g', unit='Angstrom', verbose=0)

res = xc.df_x2c_casscf(mol, ncore=4, nact=2, nele=2,
                       auxbasis='def2-universal-jkfit',
                       g_conv=1e-6, max_cycle=400)
print('E', res.e_tot, 'gnorm', res.gnorm,
      'converged', res.converged)
print('aux basis', res.auxbasis)

grad, parts = xc.df_x2c_casscf_gradient(mol, res)
print('gradient', grad)
for name, value in parts.items():
    print(f'{name:14s} {np.abs(value).max(): .3e}')
```

The four `parts` are the one-electron X2C term, the density-fitted two-electron term, the Pulay term, and the nuclear repulsion. Their sum is the gradient; the split is useful when a gradient looks wrong, because a factor-derivative error and a Pulay sign error do not have the same size.

The single-state gradient is the direct derivative at fixed orbitals and CI vector: at a converged CASSCF point both responses vanish. A state-averaged calculation is different, because the orbitals minimize the average and the target state is not stationary:

```
res_sa = xc.df_x2c_casscf(mol, ncore=4, nact=2, nele=2,
                          auxbasis='def2-universal-jkfit',
                          nroots=2, weights=[0.5, 0.5],
                          g_conv=1e-6, max_cycle=400)
print('SA energy', res_sa.e_sa, 'states', res_sa.state_energies)

g0, info = xc.df_x2c_sa_state_gradient(mol, res_sa, state=0,
                                       return_info=True)
print('state 0 gradient', g0)
print('response directions', info['n_response'])
print('retained directions', info['n_kept'])
print('Hessian spectrum', info['hess_eigvals'])
```

The state-averaged response is a finite-difference construction of the orbital Hessian and the mixed derivative around the analytic state-averaged orbital gradient. The CI vectors are re-diagonalized at every displacement, so the CI response is inside the Hessian. This is the route to use when a per-state gradient is needed; do not replace it by re-optimizing the orbitals at the displaced geometry, which on water can land in a second orbital minimum.

13.2 Two-component DF-X2C Hartree–Fock

The two-component layer keeps the spinor structure of the X2C transform. The Hamiltonian is complex Hermitian, the SCF is a complex generalized Hartree–Fock, and the density-fitted Coulomb and exchange matrices are built directly in the contracted spinor basis.

```
import numpy as np
from pyscf import gto
from pyscf.x2c import x2c as pyscf_x2c
from diffmrc.core import x2c_df

mol = gto.M(atom='O 0 0 0; H 0 -0.757 0.587; H 0 0.757 0.587',
            basis='6-31g', unit='Angstrom', verbose=0)

h = x2c_df.df_x2c_hcore(mol)
h_ref = pyscf_x2c.SpinorX2CHelper(mol).get_hcore(mol)
print('hcore max difference', np.abs(h - h_ref).max())

hf = x2c_df.df_x2c_hf(mol, auxbasis='def2-universal-jkfit')
print('E', hf.e_tot, 'gnorm', hf.gradient_norm,
      'converged', hf.converged)
print('orbital energies', hf.mo_energy[:4])

grad, parts = x2c_df.df_x2c_hf_gradient(mol, result=hf)
print('gradient', grad)
print('parts', {k: float(np.abs(v).max()) for k, v in parts.items()})
```

The one-electron check against pyscf is the first thing to run on a new basis: it isolates the X2C transform from the density fitting. If the Hamiltonian differs, the SCF and the gradient will both be wrong for a reason that has nothing to do with the two-electron part.

The spinor ERI convention is the complex-GHF one. The three-center tensor is Hermitian in the AO pair, and the derivative with respect to the second AO center carries a complex conjugate.

The test suite pins that conjugate against a geometry finite difference; a gradient that looks right at the 10^{-3} level but not at 10^{-6} usually has a missing conjugate or a wrong AO slice.

13.3 Valence-level SOMF for ECP calculations

The all-electron SOMF operator uses the bare nuclear charges and the full electron density. With an effective core potential the core electrons are absent, and the spin–orbit part of the ECP replaces their contribution. The valence-level operator is

$$h_k^{\text{SOMF-ECP}} = \frac{\alpha^2}{2}\left[h_k^{\text{nuc}} + h_k^{2e}[P]\right] - V_k^{\text{ECP}}, \tag{4}$$

with the nuclear term kept only on centers without a spin–orbit ECP, the valence $J - \frac{3}{2}K$ mean field of the spin-summed density P, and the three-component `ECPso` integral.

```
import numpy as np
from pyscf import gto
from diffmrc.core import somf_ecp

mol = gto.M(atom=[('Ge', (0.0, 0.0, 0.0)), ('F', (0.5, 0.3, 2.4))],
            basis={'Ge': 'lanl2dz', 'F': '6-31g'},
            ecp={'Ge': 'ecpds28mdfso'}, spin=1,
            unit='Bohr', verbose=0)

# A density of the electrons present in the calculation.  Here it is the
# equal-occupation reference density; a converged SCF density works too.
s = mol.intor('int1e_ovlp')
P = (mol.nelectron / mol.nao_nr()) * np.linalg.inv(s)
P = 0.5 * (P + P.T)

h_soc = somf_ecp.valence_somf_matrix(mol, P, convention='i', check=True)
print('SOC matrix shape', h_soc.shape)
print('antisymmetry', np.abs(h_soc + h_soc.transpose(0, 2, 1)).max())

# The density response is a separate contraction; the SOC kernel is not
# symmetric under (pq) <-> (rs).
m_resp = somf_ecp.somf_density_response(mol, P)
print('response shape', m_resp.shape)

print(somf_ecp.somf_definition(mol))
```

`somf_definition` records which centers contribute the ECP term, which keep the nuclear term, and the approximations that the ECP data do not fix. The operator is a prototype for relative comparisons, not a production absolute spin–orbit method: the split between the ECP spin–orbit operator and the explicit valence mean field is a fitting freedom. On the GeF model the valence-level splitting is 28.37 cm^{-1} against 43.72 cm^{-1} for the nuclear-only operator; report the variant bracket rather than a single number.

The eligibility rule is simple: the molecule must carry at least one spin–orbit ECP (`mol.has_ecp_soc()` is true). A scalar-only ECP does not supply `ECPso`, and `ecp_soc` raises. On centers without a spin–orbit ECP the nuclear term is used with the effective charge from the atom table.

13.4 Where the scalar and two-component X2C energies differ

The scalar layer and the two-component layer are not a controlled one-parameter pair: the scalar layer is CASSCF and the two-component layer is HF. At the same wavefunction level the X2C-specific difference separates into three terms,

$$E_{2c} - E_{1c} = \Delta E_{\text{spin-free}} + \Delta E_{\text{SO}} + \Delta E_{2e}, \tag{5}$$

where $\Delta E_{\text{spin-free}}$ is the change from the spin-dependent renormalization and picture-change matrix that the spin-free transform drops, ΔE_{SO} is the spin–orbit contribution, and ΔE_{2e} is the difference between the spatial DF and spinor RI-JK two-electron treatments.

For a non-degenerate closed-shell determinant the first-order spin–orbit expectation value is quenched:

$$\langle \Phi | H_{\text{SO}} | \Phi \rangle = 0 \qquad \text{(closed-shell, non-degenerate).} \tag{6}$$

The surviving ΔE_{SO} is second order and comes from the response of the orbitals and the CI, not from a direct matrix element. For an open-shell or degenerate system the first-order term is not quenched and can dominate. The measured OH/OF splittings in the manual chapter are an open-shell example; a closed-shell water comparison is dominated by the spin-free and two-electron terms.

The practical rule is: do not quote a scalar-versus-two-component energy difference as a relativistic correction unless the wavefunction method and the two-electron operator are the same on both sides. The shipped scalar CASSCF and two-component HF do not satisfy that condition.

13.5 Which layer to use

Situation	Layer
Real, spin-free multireference calculation on a full-electron basis	`core.x2c_casscf_df`
Complex one-electron spin–orbit at the HF level	`core.x2c_df`
Spin–orbit splitting of a valence configuration with an ECP	`core.somf_ecp` (relative comparisons)
Two-electron picture-change correction	`core.x2c_sap` (separate, not covered here)
ECP multireference with spin–orbit coupling	not in this release

14 RIJONX SCF: an RI-J Coulomb reference with exact exchange

The multireference chain consumes a converged SCF reference. The RIJONX driver fits the Coulomb term with the same three-center factors as the correlation layers and keeps the exchange four-center exact:

$$E = E_{\text{nuc}} + \mathrm{Tr}(Dh) + \tfrac{1}{2}\mathrm{Tr}(DJ_{\text{RI}}) - \tfrac{1}{4}\mathrm{Tr}(DK_{\text{exact}}), \qquad J_{\text{RI}} = WDW^{T}, \tag{7}$$

with $W = BS^{-1/2}$. This is RIJONX, not RI-JK and not RIJCOSX. The exchange branch is the dense four-center integral, so the method removes the Coulomb four-center work, not the exchange four-center work.

14.1 Closed-shell water

```
import os
os.environ['JAX_PLATFORMS'] = 'cpu'
for v in ('OMP_NUM_THREADS', 'MKL_NUM_THREADS',
          'OPENBLAS_NUM_THREADS', 'NUMEXPR_NUM_THREADS'):
    os.environ[v] = '4'
```

```python
import numpy as np
import jax
jax.config.update('jax_enable_x64', True)

from pyscfad import gto, scf
from diffmrc.driver.ri_scf_driver import rijonx_scf, rijonx_scf_gradient

mol = gto.Mole()
mol.atom = 'O 0 0 0.1173; H 0 0.7572 -0.4692; H 0 -0.7572 -0.4692'
mol.basis = '6-31g'
mol.verbose = 0
mol.build()

mf = rijonx_scf(mol)
ref = scf.RHF(mol)
ref.kernel()
print('RIJONX ', mf.e_tot)
print('dense  ', ref.e_tot)
print('difference', mf.e_tot - ref.e_tot)
```

The difference is the J fit, typically of order 10^{-5} Eh for this basis and auxiliary set. It is not an implementation error, and it does not shrink with SCF convergence; the SCF is converged on the RIJONX energy surface, not on the dense one.

14.2 Where the error comes from

Write the fitted Coulomb matrix as

$$J_{\mathrm{RI}} = J_{\mathrm{exact}} + \Delta J_{\mathrm{fit}}, \tag{8}$$

with the exchange matrix left exact. At a fixed density the energy error is

$$\Delta E_{\mathrm{fit}} = \tfrac{1}{2}\mathrm{Tr}\left(D\,\Delta J_{\mathrm{fit}}\right), \tag{9}$$

and no corresponding exchange term exists, because K is not fitted. At self-consistency the density also relaxes on the RIJONX surface, so the difference to a dense RHF calculation is

$$E_{\mathrm{RIJONX}} - E_{\mathrm{RHF}} = \Delta E_{\mathrm{fit}}[D_{\mathrm{RI}}] + \left(E_{\mathrm{RIJONX}}[D_{\mathrm{RI}}] - E_{\mathrm{RIJONX}}[D_{\mathrm{RHF}}]\right), \tag{10}$$

where the second term is the density-relaxation contribution. The recorded H_2O/6-31G shift is -3.75×10^{-5} Eh (`tests/driver/test_ri_scf_driver.py`); OH/6-31G gives -2.73×10^{-5} Eh and HI/def2-svp with an ECP gives -2.56×10^{-5} Eh. These are fitting errors, not implementation errors, and they should be quoted with the auxiliary basis.

The same split applies to the gradient:

$$\frac{dE}{dR} = \left(\frac{\partial E}{\partial R}\right)_{\mathrm{RI}} + \left(\frac{\partial E}{\partial D}\right)\frac{dD}{dR}, \tag{11}$$

where the first term contains the analytic RI-J factor derivative and the dense exact-K derivative, and the second is the orbital-response completion supplied by `rijonx_scf_gradient`. Dropping the second term is the fixed-coefficient error of order 1 Eh/bohr discussed below.

14.3 The converged gradient

`rijonx_scf_gradient` is the entry point for a converged reference. It combines the fixed-coefficient (skeleton) gradient with the overlap-derivative completion that the skeleton alone misses.

```
g = rijonx_scf_gradient(mf)
g_imp = np.asarray(mf.energy_grad().coords)
print('explicit', g)
print('implicit', g_imp)
print('max difference', np.abs(g - g_imp).max())
```

The fixed-coefficient functions in `driver.ri_scf` (`rijonx_energy`, `rijonx_gradient`) are not SCF gradients even at a converged density. They are useful for checking the integral and factor derivatives at fixed coefficients, and `tests/driver/test_ri_scf.py` does exactly that against finite differences. Using them as a molecular gradient misses the orbital response and is wrong at order 1 Eh/Bohr.

14.4 Open shell: ROHF and UHF

A non-zero `mol.spin` selects `RIJONX_ROHF`. Pass `uhf=True` to select `RIJONX_UHF`. The explicit gradient handles both layouts; it is not necessary to fall back to implicit differentiation.

```
mol = gto.Mole()
mol.atom = 'O 0 0 0; H 0 0 2.30'
mol.basis = '6-31g'
mol.spin = 1
mol.verbose = 0
mol.build()

mf_rohf = rijonx_scf(mol)
g_rohf = rijonx_scf_gradient(mf_rohf)
g_imp = np.asarray(mf_rohf.energy_grad().coords)
print('ROHF explicit vs implicit', np.abs(g_rohf - g_imp).max())

mf_uhf = rijonx_scf(mol, uhf=True)
g_uhf = rijonx_scf_gradient(mf_uhf)
g_imp_uhf = np.asarray(mf_uhf.energy_grad().coords)
print('UHF explicit vs implicit', np.abs(g_uhf - g_imp_uhf).max())
```

The stretched OH geometry is the useful test. At equilibrium the gradient is close to zero, and an absolute tolerance cannot distinguish a wrong response term from a small one. The recorded checks are 10^{-6} Eh/Bohr for ROHF explicit against implicit and 10^{-5} for UHF, with central differences at the same scale.

14.5 ECP references

An ECP molecule works when the molecule is built with an explicit `ecp` argument. The core-Hamiltonian helper adds the scalar ECP term, and the auxiliary basis is built from the same atom list.

```
mol = gto.Mole()
mol.atom = 'H 0 0 0; I 0 0 1.61'
mol.basis = 'def2-svp'
mol.ecp = 'def2-svp'
mol.verbose = 0
mol.build()

mf = rijonx_scf(mol)
print('HI ECP energy', mf.e_tot)
g = rijonx_scf_gradient(mf)
print('gradient', g)
```

The recorded ECP gradient deviation against a central difference is 3.0×10^{-7} Eh/Bohr on this system. The RI-J-minus-dense energy shift is -2.6×10^{-5} Eh, which is the fit.

14.6 Limits that bite in practice

- Build a fresh `Mole` and mean-field object per geometry. The RI factor cache and the pyscfad integral caches are keyed on the built molecule and are not reliably refreshed by an in-place geometry edit.
- Range-separated Coulomb is refused: the factors are built for the full operator. RI-JK and a production RIJCOSX route are out of scope; the COSX functions in `driver.ri_scf` are a fixed-grid prototype for exchange and are not a production path.
- The exact exchange branch still forms and caches the dense $\texttt{nao}^4$ tensor and, under geometry differentiation, its dense derivative. RI-J removes the Coulomb part of that work, not the exchange part.
- Fractional occupations are refused by the explicit gradient. Use integer-occupation RHF, ROHF, or UHF references.
- A non-converged SCF raises rather than returning a reference. This is deliberate: every downstream multireference number would otherwise rest on an unconverged density.